



pentest.lab

door Joost Hebly

Here I break things, repair them, and learn from the chaos — I've got my own cybersecurity subnet, a hobby project that grew into the core of my move into cybersecurity.

CEH-opgeleid

Blue Team

Red Team

Active Directory

SIEM / Detection Engineering

Incident Response

Eigen SOC-dashboard (Node.js)

```
$ whoami
```

```
naam                Joost Hebly
```

```
leeftijd            35
```

```
locatie             Amersfoort, Nederland
```

```
github              github.com/joosthebly50/dev
```

```
masterdocumentatie volledige architectuur, dagrapporten & troubleshooting-geschiedenis
```

```
werkervaring        Owner-operator internationaal transport, Nederland–Scandinavië
```

```
eerdere ervaring    4 jaar medewerker ICT-beheer, Hogeschool Utrecht
```

```
hobby               Cybersecurity — al jaren mijn grootste interesse naast het werk, nu de volgende stap
```

```
vrije tijd          Longboarden met mijn Mechelse herder, samen trainen op bijtwerk & scent detection, en gitaar spelen
```



// WAT IK ZOEK

Ondanks dat ik uit de transportwereld kom, is ethical hacking altijd mijn interesse geweest — thuis heb ik hier zelf al mee geoefend, wat uiteindelijk is uitgegroeid tot dit lab. Ik wil heel graag verder leren in een team met ervaren collega's, en zou daarnaast graag mijn OSCP en CISSP willen behalen — in combinatie met interessant en uitdagend werk.

// WAAROM DIT PROJECT

Ik heb de afgelopen jaren als zelfstandig transportondernemer gewerkt — een vak dat vraagt om zelfstandigheid, probleemoplossend vermogen en verantwoordelijkheid nemen als er iets misgaat, vaak onder tijdsdruk en zonder dat iemand het voor je oplost. Die instelling neem ik nu mee in een bewuste overstap naar cybersecurity, onderbouwd met een CEH-opleiding (Certified Ethical Hacker).

Om die kennis niet bij theorie te laten, heb ik dit lab zelf ontworpen, gebouwd, stukgemaakt en weer gerepareerd: een complete, geïsoleerde bedrijfsomgeving met een echte firewall, een Windows-domein, een SOC-platform en een aanvalswerkstation — inclusief alle storingen die in een echte omgeving ook voorkomen. Dit project is de manier waarop ik laat zien wat ik kan, niet alleen wat ik heb geleerd.

// WAT DIT PROJECT IS

pentest.lab is een privé cybersecurity-oefenomgeving, volledig gevirtualiseerd op eigen hardware. Het bestaat uit een firewall/gateway, een Windows-domeincontroller, een SOC-platform voor detectie en monitoring, een aanvalswerkstation en meerdere losse, bewust kwetsbare oefendoelen (Metasploitable2, OWASP Juice Shop, DVWA) — allemaal op een geïsoleerd netwerk, met verkeer dat automatisch wordt gespiegeld naar het SOC-platform voor analyse.

Het startpunt was een eigen idee ("Fortress Bazzite"): live meekijken wat er op mijn netwerk gebeurt, direct een melding krijgen zodra iets verdachts gebeurt, en uiteindelijk automatisch kunnen reageren op een aanval. Die visie draait nu op Security Onion — dat monitoring, detectie (Suricata/Zeek) en alerting al gebundeld aanbiedt — aangevuld met Elastic Agent/Fleet voor host-monitoring. Live zicht en directe alerts staan overeind; automatische respons is de volgende stap.

Het netwerk staat nu bewust nog in zijn *eerste fase*. Het doel is een omgeving waarin ieder type aanval gesimuleerd kan worden — en er direct de bijpassende detectie en verdediging tegenover staat. Elke volgende uitbreiding voegt zowel een aanvalstechniek als de manier om die te herkennen toe.

Naast Security Onion heb ik er inmiddels ook zelf een eigen SOC-dashboard bovenop gebouwd (Node.js): live systeemgezondheid van de hostmachine, een paneel met actieve netwerkverbindingen (PID, proces, 1s-refresh), een alertfeed die Suricata-signalen indeelt in 15 aanvalscategorieën met severity-badges en herkenbare hostnamen, en gesproken spraakalerts (tekst-naar-spraak) met prioriteits- en cooldown-logica zodat het systeem niet spamt bij een aanhoudende scan. Dit dashboard is geen theoretische oefening: het is live getest tegen een echte exploit (zie hieronder).

RED TEAM / OFFENSIEF

Metasploit Framework

Exploitatie tegen kwetsbare doelsystemen

Actief

Burp Suite

Webapplicatie-aanvallen (Juice Shop / DVWA)

Actief

Aircrack-ng

Wifi-monitor mode, via PCI-passthrough
wifikaart

Gepland

WiFi Pineapple

Draadloze aanvalssimulaties, apart
gesegmenteerd

Gepland

BloodHound / Impacket

Active Directory-aanvalspaden in kaart
brengen

Gepland

Hashcat

Wachtwoord-cracking op gecontroleerde
hashes

Gepland

BLUE TEAM / DEFENSIEF

Wireshark

Packet-analyse van gespiegeld labverkeer

Actief

Security Onion / Suricata / Zeek

Netwerkdetectie en -telemetrie

Actief

Elastic Fleet / Sysmon

Host-based monitoring op endpoints

Actief

Eigen SOC Alarmdashboard

Node.js: health bar, live verbindingen, 15
aanvalscategorieën, spraakalerts

Actief

Alert-triage agents (lokaal + AI)

Automatische false-positive filtering bij ingest,
AI-escalatie voor twijfelgevallen

Actief

Wazuh

File integrity monitoring, rootkit-detectie

Gepland

MITRE ATT&CK-mapping

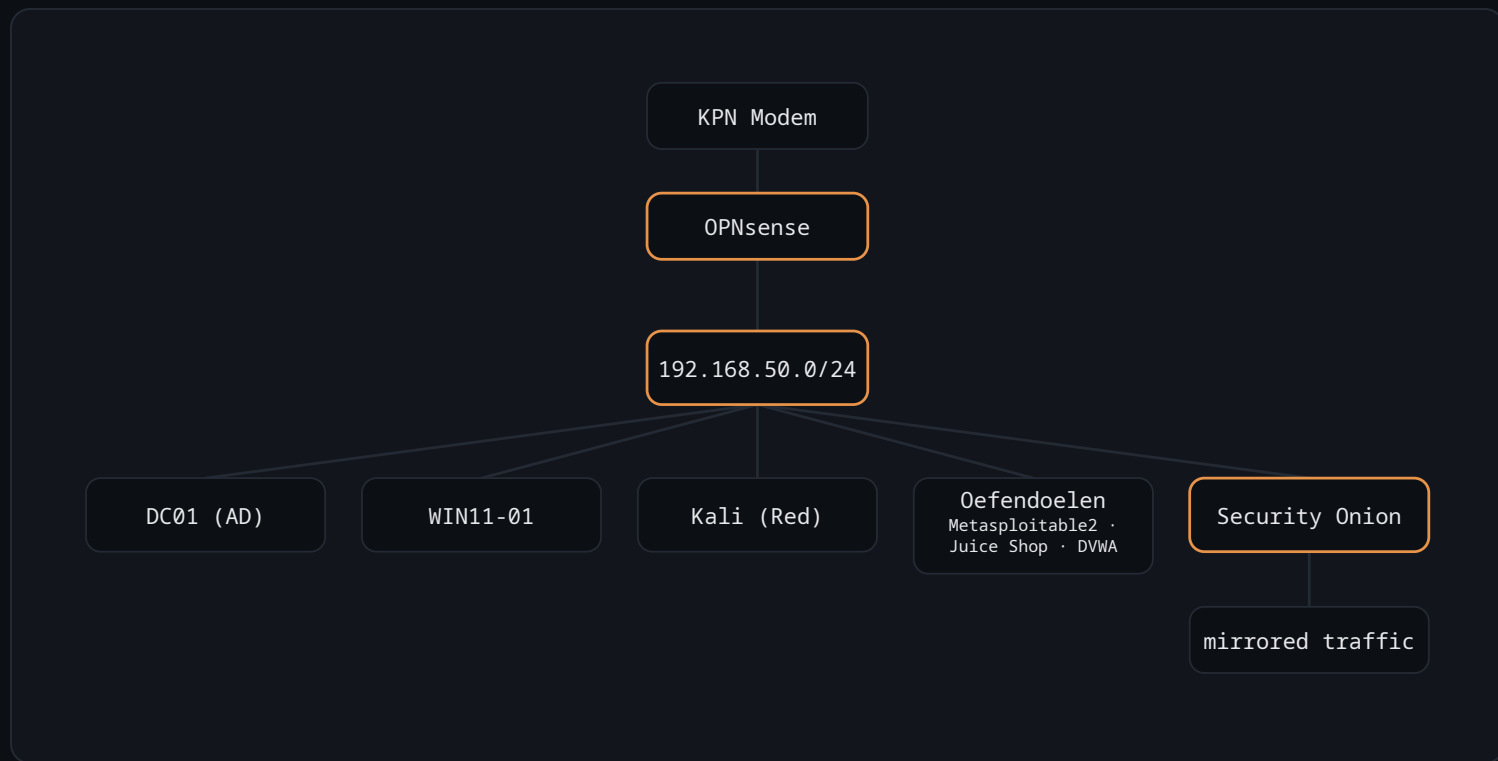
Elke aanval koppelen aan een herkende
techniek

Gepland

Honeypots

Vroege detectie van laterale beweging

Gepland



Netwerkbeveiliging & firewalls

Firewall-architectuur op twee niveaus (OPNsense + Security Onion's eigen hostgroup-model), root-cause-analyse van geblokkeerd verkeer via directe pakketeventidentie in plaats van aannames.

SIEM & detection engineering

Security Onion / Elastic Fleet beheer, Sysmon-uitrol met een geverifieerde community-configuratie, end-to-end validatie van logstromen tot en met de daadwerkelijke Elasticsearch-index.

Windows Server & Active Directory

Domeincontroller-inrichting, DNS-integratie, tijdsynchronisatieproblemen op een PDC-emulator gediagnosticeerd tot op hypervisor-niveau.

Linux & virtualisatie

KVM/QEMU/libvirt-beheer, event-driven automatisering (geen polling), netwerkisolatie en traffic mirroring.

Incident response-methodiek

Elke conclusie onderbouwd met bewijs (logs, pakketten, herhaalde tests), nooit aannames — inclusief het expliciet testen of een fix een herstart overleeft, niet alleen of hij op dat moment werkt.

Documentatiediscipline

Volledige, reproduceerbare documentatie: architectuur, dagrapporten, commandologs, troubleshooting-geschiedenis en rollback-procedures voor elke wijziging.

Eigen SOC-tooling bouwen (full-stack)

Zelf een SOC-dashboard ontworpen en gebouwd (Node.js): live host-metrics, een actieve-verbindingenpaneel, categorisatie van Suricata-alerts en spraakgestuurde alerting met prioriteits-/cooldown-logica — vervolgens live gevalideerd tegen een echte exploit in plaats van alleen synthetisch testverkeer.

Alert-triage agents (lokaal + AI)

Twee samenwerkende triage-agents gebouwd voor het Alarmdashboard: een lokale, deterministische regel-engine (signature-kennisbank, known-process- en timing-correlatie) die automatisch op elk binnenkomend alert draait vóórdat het zichtbaar wordt, plus een AI-agent (Claude Code) die twijfelgevallen in een wachtrij oppakt voor diepere analyse. Vaste veiligheidsregel: hoge-severity categorieën (REVERSE_SHELL, PRIV_ESC, EXPLOIT, CRED_ACCESS, LATERAL_MOVEMENT, PERSISTENCE, MITM, SQLI, XSS) worden nooit automatisch aangeraakt.

Eén storing, drie onafhankelijke oorzaken

Windows-domeincontroller · Elastic Fleet · Security Onion firewall · tijdsynchronisatie

SITUATIE

De domeincontroller stond op "Offline" in het SIEM. Geen Windows-, Sysmon- of endpoint-telemetrie kwam binnen, terwijl de achterliggende Windows-diensten gewoon draaiden — geen crash, maar een stille storing ergens in de keten tussen host en SIEM. Dit is één representatief voorbeeld van hoe troubleshooting in dit lab wordt aangepakt.

AANPAK

Systematisch, laag voor laag, elke conclusie onderbouwd met bewijs in plaats van aannames: firewall-logs, poorttests, Fleet-API-responses, Logstash-pipelinestatistieken en Elasticsearch-foutmeldingen. Bij elke fix: opnieuw testen, inclusief een volledige herstart van zowel de domeincontroller als het SIEM-platform.

DRIE GESTAPELDE, ONAFHANKELIJKE ROOT CAUSES

01 Onvolledige firewall-toegang

De host zat wel in de groep voor webtoegang, maar niet in de groepen die toegang geven tot de SIEM-agentpoorten — drie afzonderlijke poorten moesten worden vrijgegeven.

02 Klokverschil van 9 uur, kwam terug na elke herstart

Een hypervisor-tijdsynchronisatiedienst overschreef de correcte NTP-tijd bij elke boot — een subtiele bug die ook stilletjes logdata liet verdwijnen door tijdstempels die in de toekomst lagen.

03 Ontbrekende telemetriebron

Het SIEM verwachtte al langer gedetailleerde eventdata die simpelweg nog nooit was geïnstalleerd — pas zichtbaar nadat de eerste twee lagen waren opgelost.

RESULTAAT, GEVERIFIEERD

Overleeft agent-herstart

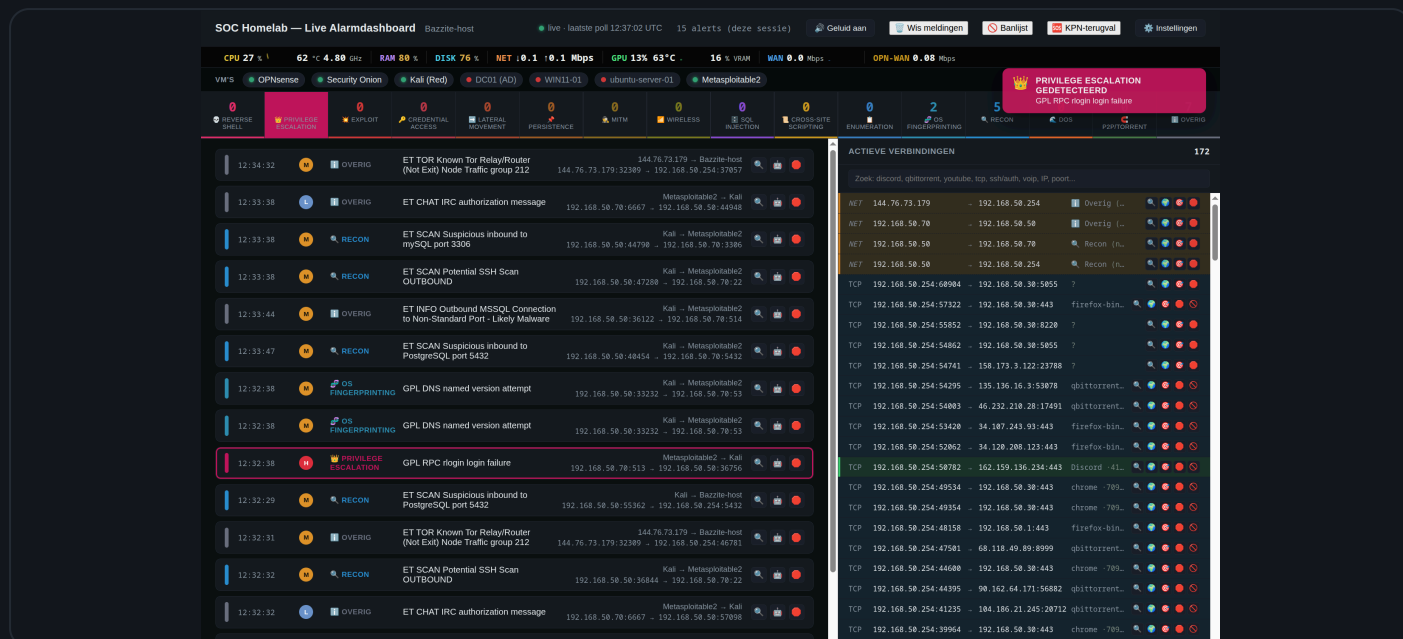
Overleeft domeincontroller-reboot (2×)

Overleeft SIEM-reboot

Volledige telemetrie bevestigd end-to-end

vsftpd 2.3.4-backdoor (CVE-2011-2523) tegen het eigen SOC-dashboard

Kali (Red) → Metasploitable2 · SOC Alarmdashboard · Security Onion / Suricata



Live screenshot – SOC Alarmdashboard tijdens een netwerkbrede recon-scan: nieuw VM-statuspaneel (rood/groen), PRIVILEGE ESCALATION-banner (GPL RPC rlogin login failure), RECON-/OS-FINGERPRINT-alerts en 172 actieve verbindingen

SITUATIE

Om het eigen SOC Alarmdashboard niet alleen tegen synthetisch testverkeer te valideren, maar tegen een echte aanval, is de bekende vsftpd 2.3.4-achterdeur opnieuw uitgevoerd vanaf Kali tegen Metasploitable2: een Nmap-servicescan gevolgd door de FTP-trigger `USER x: )` en een read-only root-shell (`id/whoami/uname/hostname`) via TCP/6200.

RESULTAAT, LIVE GEVERIFIEERD

- De scan leverde 16 RECON-alerts op in de teller van het dashboard.
- De trigger zelf werd niet gedetecteerd — realistisch: die eerste stap gaf Suricata weinig om op te haken — maar de root-shell activeerde GPL ATTACK_RESPONSE id check returned root, correct ingedeeld als PRIV_ESC.
- Het verbindingenpaneel toonde een rode synthetische "NET"-rij voor het verkeer tussen de twee labmachines (dat de hostmachine zelf nooit passeert), plus de eigen ssh-sessie die secundair oplichtte omdat Kali zelf als verdachte host werd herkend.

DISCIPLINE

Voor Tier 2/3-exploitatie geldt een vaste regel: altijd een snapshot vóór de test, en de omgeving daarna aantoonbaar weer schoon. Bij deze test bleven geen bestanden, gebruikers of sessies achter op het doelsysteem (geverifieerd via `ss -tn`), dus was geen herstel nodig.

16 RECON-alerts correct geteld

PRIV_ESC correct herkend op root-shell-output

Synthetische NET-rij correct getoond

Doelsysteem geverifieerd schoon achtergelaten

// TIJDLIJN

Fase 1

Fortress Bazzite — het startpunt

Eerste aanvals-/doelomgeving, exploitatie geoefend op bewust kwetsbare systemen, eigen Suricata-detectie rechtstreeks op de hostmachine.

Fase 2

Firewall & netwerkfundament

Eigen firewall/gateway geïnstalleerd, netwerksegmentatie en DHCP/DNS-architectuur uitgewerkt en herhaaldelijk gedebugd tot stabiel.

Fase 3

Identity & SOC-platform

Windows-domeincontroller opgezet; SOC/SIEM-platform geïnstalleerd en gekoppeld aan het domein via Elastic Fleet.

Fase 4

Automatisering & operationele volwassenheid

Event-driven traffic mirroring, browsergestuurde SOC-operator-workflow, desktop-tooling voor dagelijks gebruik.

Fase 5

De DC01-case

Diepgaande, bewijsgedreven troubleshooting van een drielaags storing tot volledige, herhaaldelijk geverifieerde stabiliteit — zie case study hierboven.

Fase 6

Eigen SOC Alarmdashboard & eerste live aanvalstests

Zelf een SOC-dashboard gebouwd bovenop Security Onion (Node.js): systeemgezondheid, live actieve verbindingen, 15 aanvalscategorieën met severity-badges en spraakalerts. Vervolgens live getest tegen een echte exploit (vsftpd 2.3.4-backdoor op Metasploitable2) met vaste snapshot-/opruimdiscipline rond elke Tier 2/3-test.

Fase 7

Alert-triage agents: lokaal + AI, samen

Twee samenwerkende triage-agents toegevoegd aan het Alarmdashboard: een lokale, deterministische regel-engine die nu automatisch op elk binnenkomend alert draait vóórdat het zichtbaar wordt in de feed, plus een AI-agent (Claude Code) die twijfelgevallen in een wachtrij oppakt voor diepere analyse. Vaste veiligheidsregel: hoge-severity categorieën worden nooit automatisch aangeraakt. Live geverifieerd: herhaalde ET TOR/torrent-coïncidentie-ruis volledig verdwenen uit de feed, over meerdere pollcycli.

// TECHNOLOGIEËN

KVM / QEMU / libvirt

OPNsense

Security Onion

Suricata

Zeek

Elastic Stack (Elasticsearch, Kibana, Fleet, Logstash)

Sysmon

Windows Server / Active Directory

Kali Linux

Bash / PowerShell

systemd

Git

Node.js

Playwright

Piper TTS

Nmap / Metasploit Framework

Burp Suite

Wireshark

Aircrack-ng

Reaver

Social-Engineer Toolkit (SET)

// ACHTERGROND

4 jaar ICT-beheer — Hogeschool Utrecht

Voordat ik zelfstandig ondernemer werd, heb ik 4 jaar als medewerker ICT-beheer gewerkt bij Hogeschool Utrecht: dagelijkse ervaring met systeem- en netwerkbeheer en gebruikersondersteuning binnen een grote, professionele IT-omgeving — een directe basis voor de technische kant van dit lab.

CEH — Certified Ethical Hacker

Formele basis voor de aanvalsgerichte kant van dit lab: methodisch enumereren, kwetsbaarheden herkennen en gecontroleerd exploiteren binnen een geïsoleerde, eigen omgeving.

Jarenlange ervaring als zelfstandig ondernemer

Zelfstandigheid, verantwoordelijkheid nemen bij storingen, en onder druk zelf een probleem tot op de bodem uitzoeken — precies de instelling die dit lab vereiste.

Op weg naar OSCP & CISSP

Dit lab is tegelijk mijn oefenterrein richting OSCP (praktische exploitatievaardigheid) en de bredere beveiligingsprincipes waar CISSP om vraagt — niet losstaand geleerd, maar direct toegepast in een omgeving die ik zelf beheer.

"VMs break. Onions have
layers. Firewalls judge your
decisions. Coffee disappears
mysteriously."

— het motto op de muur van mijn eigen SOC, letterlijk

pentest.lab is een doorlopend, zelfstandig project — gebouwd, stukgemaakt, gerepareerd en gedocumenteerd door Joost Hebly, als onderbouwing bij de overstap naar een rol in cybersecurity.

Graag licht ik ook de uitgebreide documentatie van mijn project aan u toe — architectuur, dagrapporten, commandologs en troubleshooting-geschiedenis zijn allemaal volledig vastgelegd.